

LAPORAN PRAKTIKUM STRUKTUR DATA

“BINARY TREE DAN GRAPH”

Disusun Oleh :

Dzhillan Dzhalila

2511531001

Dosen Pengampu:

Dr. Wahyudi, S.T, M.T

Asisten Praktikum:

Muhammad Galid A.



FAKULTAS TEKNOLOGI INFORMASI

DEPARTEMEN INFORMATIKA

UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji syukur penulis panjatkan atas kehadiran Allah SWT Tuhan Yang Maha Esa yang telah memberikan rahmat dan berkatnya, sehingga penulis dapat menyelesaikan laporan praktikum ini yang telah dilaksanakan sebagai pertemuan ke-9 untuk mata kuliah Praktikum Struktur Data. Praktikum ini berfokus pada pembahasan konsep struktur data Binary Tree dan Graph.

Melalui praktikum ini, hasil yang diharapkan adalah penulis dapat memahami dan mengimplementasikan penggunaan penelusuran dari konsep binary tree dan juga konsep Graph. Dengan proyek proyek sederhana, diharapkan penulis dapat untuk memahami logika dibalik program yang menggunakan konsep Binary Tree dan Graph.

Penulis menyadari bahwa, laporan praktikum ini masih memiliki banyak kekurangan. Oleh karena itu, kritik dan saran yang membangun sangat diharapkan demi perbaikan dan pengembangan di masa mendatang. Semoga laporan ini dapat memberikan manfaat, baik sebagai bahan pembelajaran penulis maupun referensi pembaca dalam memahami dasar-dasar pemrograman.

Padang, 2026

Dzhillan Dzhalila

DAFTAR ISI

KATA PENGANTAR.....	i
DAFTAR ISI	ii
BAB I PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Tujuan.....	1
1.3 Manfaat	1
BAB II PEMBAHASAN	2
2.1 Dasar Teori	2
2.1.1 Binary tree.....	2
2.2 Program Node_2511531001	3
2.2.1 Kode Program	3
2.3 Program BTree_2511531001	7
2.3.1 Kode Program	7
2.4 Program BTreeDriver_2511531001	11
2.4.1 Kode Program	11
2.4.2 Output.....	14
2.5 Program GraphTraversal_2511531001	14
2.5.1 Kode Program	14
2.5.2 Output.....	19
BAB III KESIMPULAN.....	20
DAFTAR PUSTAKA	21
LAPORAN TUGAS.....	22

BAB I

PENDAHULUAN

1.1 Latar Belakang

Pada ilmu komputer dan pengembangan perangkat lunak, kita tak hanya bergantung pada algoritma program yang efisien, namun juga harus bisa memilih struktur data untuk menyimpan dan mengorganisir data dengan tepat. Binary Tree dan Graph adalah dua struktur data non linear yang dapat digunakan untuk data yang memiliki hubungan hierarkis atau berbentuk jaringan atau network. Binary Tree biasanya digunakan untuk mengurus data data yang memiliki hubungan hierarkis dimana setiap node dapat memiliki anak (left child dan right child) dan anak anak tersebut dapat memiliki anak kembali. Struktur data ini berperan penting dalam operasi komputer seperti pencarian data, pengurutan data dan juga representasi struktur direktori file. Graph adalah generalisasi dari binary tree yang terdiri dari simpul yaitu vertex dan juga edges yaitu sisi yang menghubungkan dua simpul. Penerapan graph di dunia nyata adalah pada peta navigasi (untuk menentukan rute terpendek), jaringan sosial atau topologi jaringan komputer.

1.2 Tujuan

1. Mengimplementasikan struktur data Binary Tree dan Graph menggunakan bahasa pemrograman Java secara tepat.
2. Mengidentifikasi perbedaan dari algoritma penelusuran seperti inorder, preorder, postorder serta DFS (Depth First Search) dan juga BFS (Breadth First Search).

1.3 Manfaat

1. Memperdalam pemahaman konseptual mengenai cara kerja struktur data non linear dalam program.
2. Memberikan fondasi yang kuat untuk merancang solusi algoritma yang optimal dalam pengembangan perangkat lunak.

BAB II

PEMBAHASAN

2.1 Dasar Teori

2.1.1 Binary tree

Binary Tree adalah struktur data hierarkis yang terdiri dari node node yang saling terhubung. Node node yang ada pada pohon biner memiliki dua tipe node anak, yaitu anak kiri dan anak kanan. Masing masing node anak tersebut akan memiliki anak anak lagi dan begitu seterusnya. Kumpulan node ini akan membentuk suatu struktur bercabang yang mirip dengan cabang cabang pohon.

Binary tree memiliki ciri ciri mendasar antara lain adalah

1. Memiliki node, node adalah elemen yang ada dalam pohon biner. Node menyimpan data dan memiliki dua bagian utama yaitu data dan pointer. Pointer adalah referensi atau alamat yang menunjuk ke anak kiri dan kanan, jika ada.
2. Memiliki root, root adalah node pertama dalam pohon yang menjadi titik awal untuk mencapai node lainnya.
3. Memiliki Anak kiri dan atau anak kanan.
4. Tinggi pohon, yaitu berapa banyak level node yang dimiliki oleh pohon biner.
5. Adanya leaf node, leaf node adalah node yang tidak memiliki anak dan biasanya menjadi titik akhir dalam jalur dari root.
6. Memiliki subtree, yaitu setiap node dalam pohon biner beserta anak-anaknya membentuk sebuah subtree.

Dalam berbagai aplikasi komputer, binary tree dapat digunakan untuk menyimpan data ataupun melakukan pencarian data.

2.1.2 Graph

Graph adalah jenis struktur data yang terdiri dari kumpulan simpul berhingga untuk menyimpan data dan antara dua buah simpul terdapat hubungan yang saling berkaitan. Simpul pada graph sering disebut dengan vertex, sedangkan sisi yang menghubungkan antara vertex (V) disebut dengan edge (E). Kegunaan Graph dalam komputasi diantaranya adalah untuk pemodelan grafik dan digunakan dalam jaringan komputer aplikasi Peer to peer.

2.2 Program Node_2511531001

2.2.1 Kode Program

```
package Pekan9_2511531001;
public class Node_2511531001 {
    int data_1001; // pendeklarasian untuk data dan tipe
    datanya bisa diganti dengan string
    Node_2511531001 left_1001; // mendeklarasikan pointer
    untuk data yang berada di kiri
    Node_2511531001 right_1001; //mendeklarasikan
    pointer untuk data yang berada di kanan

    public Node_2511531001 (int data_1001) { //
    konstruktor variabel data dan pointer
        this.data_1001= data_1001;
        left_1001 = null;
        right_1001 = null;
    }

    public void setLeft_1001 (Node_2511531001 node_1001)
    { // setter untuk variabel pointer Left
        if (left_1001 == null) // jika pointer left
        masih kosong
            left_1001 = node_1001; //maka node akan
            ditetapkan sebagai nilai dari pointer left
        }

    public void setRight_1001 (Node_2511531001
    node_1001) { // setter untuk variabel pointer Right
```

```

        if (right_1001 == null) //jika pointer Right
        masih kosong

            right_1001 = node_1001; // maka node
            akan ditetapkan sebagai nilai dari pointer right
        }

        public Node_2511531001 getLeft_1001 () { //getter
        untuk pointer Left

            return left_1001; //mengembalikan nilai yang
            tersimpan pada pointer left, saat getLeft dipanggil
        }

        public Node_2511531001 getRight_1001 () { //getter
        untuk pointer Right

            return right_1001; //mengembalikan nilai yang
            tersimpan pada pointer right, saat getRight dipanggil
        }

        public int getData_1001 () { //getter untuk
        mengambil nilai data yang bertipe integer

            return data_1001; //mengembalikan nilai data
            saat getData dipanggil
        }

        public void setData_1001 (int data_1001) { // setter
        untuk menetapkan nilai variabel data

            this.data_1001 = data_1001;

        }

        void printPreorder_1001 (Node_2511531001 node_1001)
        { // method untuk menelusuri data dengan method preorder
            // method ini akan melakukan penelusuran
            dengan mengunjungi root, left node lalu baru right node

            if (node_1001 == null) // perbangan untuk
            mengembalikan program ke pemanggilan sebelumnya saat node
            yang dikunjungi sudah kosong

                return ;

            System.out.print(node_1001.data_1001 + "
"); // menelusuri root dari binary tree terlebih dahulu
            (data_100 adalah node yang sedang aktif)

            printPreorder_1001 (node_1001.left_1001); //
            menelusuri left subtree

```

```

        printPreorder_1001 (node_1001.right_1001); //
        menelusuri right subtree
    }

    void printPostorder_1001 (Node_2511531001 node_1001)
    { // method untuk menelusuri data dengan method postorder
        // method ini melakukan penelusuran dengan
        menelusuri left subtree lalu right subtree sebelum
        mengunjungi root
        if (node_1001 ==null) // percabangan untuk
        memberhentikan method,
            //jika node yang dikunjungi sudah
            kosong maka akan kembali ke pemanggil sebelumnya
            return ;
            printPostorder_1001 (node_1001.left_1001); //
            menelusuri seluruh subtree sebelah kiri terlebih dahulu
            sampai menemukan node null
            printPostorder_1001 (node_1001.right_1001);
            // menelusuri seluruh subtree sebelah kanan
            System.out.print(node_1001.data_1001 + " ");
            // menelusuri root
        }

        void printInorder_1001 (Node_2511531001 node_1001)
        { // method untuk menelusuri data dengan method inorder
            // method ini akan melakukan penelusuran
            dimulai dengan left subtree lalu root dan terakhir akan
            mengunjungi right subtree dari data
            if (node_1001 ==null) // penelusuran akan
            berhenti jika node yang ditelusuri sudah bernilai null
            return ; // mengembalikan program ke
            pemanggilan sebelumnya jika percabangan bernilai true
            printInorder_1001 (node_1001.left_1001); //
            menelusuri node yang terdapat pada left subtree
            System.out.print(node_1001.data_1001 + " ");
            // menelusuri root dari data atau node yang sedang aktif
            printInorder_1001 (node_1001.right_1001); //
            menelusuri node yang terdapat pada right subtree
        }
    }
}

```

```

    }

    public String print_1001() {
        return this.print_1001("",true,""); // method
        ini memberikan nilai default kepada variabel
        // prefix_1001 = "", yang artinya belum ada
        identasi
        // isTail_1001 = true, artinya node root
        dianggap sebagai 'ekor'
        //sb_1001= "", artinya belum ada identasi
    }

    // method dibawah ini digunakan untuk menggambarkan
    binary tree dengan penggambaran visual sederhana
    public String print_1001(String prefix_1001, boolean
    isTail_1001, String sb_1001) {
        // method ini akan menampilkan node dari
        kanan terlebih dahulu untuk menggambarkan visualisasi
        dengan urutan dengan terbalik
        if (right_1001 !=null) { // percabangan ini
        akan bernilai true jika pointer right tidak mengarahkan
        pada node yang kosong
            right_1001.print_1001(prefix_1001 +
            (isTail_1001 ? "|  " : "  "), false,sb_1001);
            /* jika percabangan benar, maka program
            akan menampilkan node kanan dengan secara rekursif isTail
            bernilai false
            * karena node kanan belum mencapai
            "ujung" dari penelusuran*/
        }
        System.out.println(prefix_1001 +
        (isTail_1001 ? "\\--" : "/--") + data_1001);
        //perintah untuk mencetak node yang aktif
        saat ini

        if (left_1001 !=null) {// percabangan ini
        bernilai true jika pointer left tidak mengarah ke node yang
        bernilai kosong

```

```

        left_1001.print_1001(prefix_1001 +
(isTail_1001 ? "  " : "|  "), true, sb_1001);

        /* jika percabangan benar, maka program
akan menampilkan node kiri secara rekursif dengan isTail
bernilai true

        * karena node kiri merupakan ujung
dari penelusuran dalam proses ini*/

    }

    return sb_1001; // mengembalikan nilai dari
parameter sb_1001
}
}

```

Kode Program 2.1 : Program yang mendeklarasikan node.

*Penjelasan berupa komentar pada program

2.3 Program BTree_2511531001

2.3.1 Kode Program

```

package Pekan9_2511531001;

public class BTree_2511531001 {

    private Node_2511531001 root_1001; // deklarasi dari
variabel root atau data awal dari penelusuran

    private Node_2511531001 currentNode_1001; //
deklarasi variabel dari node yang sedang dikunjungi atau
sedang aktif

    public BTree_2511531001 () { // konstruktor untuk
kelas BTree

        root_1001 = null; // dengan kondisi nilai
awal root adalah kosong atau null

        // ini berarti binary tree yang baru dibuat
benar benar dalam kondisi kosong

    }

    public boolean search_1001 (int data_1001) { //
method luaran untuk melakukan pencarian data

        /*method ini akan bernilai true jika
data_1001 berhasil ditemukan dalam pohon

```

```

        *dan akan bernilai false jika data_100 tidak
        berhasil ditemukan dalam phon
    */

    return search_1001 (root_1001, data_1001); //
    meneruskan tugas pencarian ke method search_1001 yang lebih
    lanjut

    // root_1001 ditetapkan sebagai nilai awal
    dari pencarian
}

private boolean search_1001 (Node_2511531001
node_1001 , int data_1001) { // method untuk mencari data
node tertentu di dalam pohon

    if (node_1001.getData_1001() == data_1001)//
    memeriksa node yang sedang dikunci apakah bernilai sama
    dengan nilai yang dicari

        return true;

        //jika sama persis, maka program akan
        langsung mengembalikan nilai true dan proses pencariia di
        cabang tsb dianggap selesai

        if(node_1001.getLeft_1001() != null)
        //memeriksa apakah anak node kiri tidak kosong
            if
            (search_1001(node_1001.getLeft_1001(),data_1001))//jika
            tidak kosong, maka akan dilakukan pencarian pada anak node
            kiri

                return true;

                //jika menemukan data yang
                dicari, maka program akan mengembalikan nilai true

                if(node_1001.getRight_1001() !=
                null)//memeriksa apakah anak node kanan kosong
                    if
                    (search_1001(node_1001.getRight_1001(),data_1001))//jika
                    tidak kosong maka akan dilakukan pencarian pada anak node
                    kanan

                        return true;

```

```

        //jika menemukannya pada anak
        kanan, maka program akan mengembalikan nilai true dan
        menghentikan penelusuran
        return false;

        //program akan mengembalikan nilai false jika
        node yang diinginkan tidak ditemukan pada anak kiri dan
        kanan
    }

    public void printInorder_1001 () { //method ini akan
    memanggil method printInorder yang dimiliki oleh objek
    root_1001

        root_1001.printInorder_1001(root_1001);
        //lalu melempar root_1001 sebagai parameter
        awal untuk memulai rantai rekursi
    }

    public void printPreorder_1001 () { //method ini akan
    memanggil method printPreorder yang dimiliki oleh objek
    root_1001

        root_1001.printPreorder_1001(root_1001);
        //lalu melempar root_1001 sebagai parameter
        awal untuk memulai rantai rekursi
    }

    public void printPostorder_1001 () { //method ini
    akan memanggil method printPostorder yang dimiliki oleh
    objek root_1001

        root_1001.printPostorder_1001(root_1001);
        //lalu melempar root_1001 sebagai parameter
        awal untuk memulai rantai rekursi
    }

    public Node_2511531001 getRoot_1001 () { // method
    untuk mendapatkan nilai dari root dari suatu pohon
        return root_1001; // dan mengembalikan nilai
    yang tersimpan pada variabel root_1001
    }

```

```

        public boolean isEmpty_1001 () { // mengembalikan
nilai true jika root_100 bernilai kosong atau null
        return root_1001 == null;
    }

    public int countNodes_1001 () { // method yang akan
menghitung node yang sudah dikunjungi dimulai dari
root_1001 sampai data akhir
        return countNodes_1001 (root_1001);
    }

    private int countNodes_1001 (Node_2511531001
node_1001) {
        int count_1001 = 1; // nilai awal dari
count_1001 adalah 1
        if (node_1001 == null){ // lalu jika
node_1001 tidak ada maka kembalikan nilai 0
            return 0;
        } else { // atau jika node_1001 tidak null
maka

            count_1001 += countNodes_1001
(node_1001.getLeft_1001()); //menelusuri node anak kiri dan
jumlah nodenya ditambahkan kepada variabel count_1001

            count_1001 += countNodes_1001
(node_1001.getRight_1001()); //menelusuri node anak kanan
dan jumlah nodenya ditambahkan kepada variabel count_1001
            return count_1001; // lalu mengembalikan
akumulasi keseluruhan dari node node yang ada
        }
    }

    public void print_1001 () { //memanggil method untuk
menampilkan visualisasi dari pohon
        root_1001.print_1001();
    }

```

```

        public Node_2511531001 getCurrent_1001 () { //method
        untuk mengambil nilai dari referensi variabel
        currentNode_1001

            return currentNode_1001;

        }

        public Node_2511531001 setCurrent_1001
        (Node_2511531001 node_1001) { //method untuk mengubah nilai
        currentNode_1001

            return currentNode_1001 = node_1001;

        }

        public void setRoot_1001 (Node_2511531001 root_1001)
        { //method ini mengizinkan penggantian total akar pohon dari
        luar kelas

            this.root_1001 = root_1001;

        }

    }

```

Kode Program 2.2 : Program yang mendeklarasikan Binary Tree.

*Penjelasan berupa komentar dalam program

2.4 Program BTreeDriver_2511531001

2.4.1 Kode Program

```

package Pekan9_2511531001;

public class BTreeDriver_2511531001 {

    public static void main (String [] args) {

        //membuat pohon

        BTree_2511531001 tree_1001 = new
        BTree_2511531001 ();

        System.out.print("Jumlah Simpul Awal Pohon :
        ");

        System.out.println(tree_1001.countNodes_1001()); //
        memanggil method countNodes untuk menampilkan jumlah awal
        node pohon

        //menambahkan simpul data 1
    }
}

```

```

Node_2511531001 root_1001 = new
Node_2511531001 (1);

//menjadikan simpul 1 sebagai root
tree_1001.setRoot_1001(root_1001);
System.out.print("Jumlah simpul jika hanya
ada root :");

System.out.println(tree_1001.countNodes_1001());//
menampilkan jumlah simpul yang mana pada saat ini hanya
root yang dihitung

Node_2511531001 node2_1001 = new
Node_2511531001 (2); // menambahkan nilai untuk dijadikan
node //

Node_2511531001 node3_1001 = new
Node_2511531001 (3);

Node_2511531001 node4_1001 = new
Node_2511531001 (4);

Node_2511531001 node5_1001 = new
Node_2511531001 (5);

Node_2511531001 node6_1001 = new
Node_2511531001 (6);

Node_2511531001 node7_1001 = new
Node_2511531001 (7);

Node_2511531001 node8_1001 = new
Node_2511531001 (8);

Node_2511531001 node9_1001 = new
Node_2511531001 (9);

root_1001.setLeft_1001(node2_1001);//menjadikan
node2 sebagai anak kiri dari root

node2_1001.setLeft_1001(node4_1001);//menjadikan
node4 sebagai anak kiri dari node2

node2_1001.setRight_1001(node5_1001);//menjadikan
node5 sebagai anak kanan dari node2

```

```

        node4_1001.setRight_1001(node8_1001); //menjadikan
node8 sebagai anak kanan dari node4

        root_1001.setRight_1001(node3_1001); //menjadikan
node3 sebagai anak kanan dari root

        node3_1001.setLeft_1001(node6_1001); //menjadikan
node6 sebagai anak kiri dari node3

        node3_1001.setRight_1001(node7_1001); //menjadikan
node7 sebagai anak kanan dari node3

        node6_1001.setLeft_1001(node9_1001); //menjadikan
node9 sebagai anak kiri dari node6

        //set root

        tree_1001.setCurrent_1001(tree_1001.getRoot_1001());
        System.out.println("menampilkan simpul
terakhir :");

        System.out.println(tree_1001.getCurrent_1001().getDa
ta_1001());

        System.out.println("Jumlah simpul ; setelah
simpul 7 ditambahkan :");

        System.out.println(tree_1001.countNodes_1001());
        System.out.println("InOrder:"); //menampilkan
hasil penelusuran dengan method inorder
        tree_1001.printInorder_1001();
        System.out.println("\nPreOrder:");
//menampilkan hasil penelusuran dengan method preorder
        tree_1001.printPreorder_1001();

        System.out.println("\nPostOrder:"); //menampilkan
hasil penelusuran dengan method postorder
        tree_1001.printPostorder_1001();

```

```

        System.out.println("\nmenampilkan simpul
dalam bentuk phon : "); //menampilkan tampilan visual dari
pohon

        tree_1001.print_1001();
    }
}

```

Kode Program 2.3 : Program Driver Binary Tree

2.4.2 Output

```

Jumlah Simpul Awal Pohon : 0
Jumlah simpul jika hanya ada root :1
menampilkan simpul terakhir :
1
Jumlah simpul ; setelah simpul 7 ditambahkan :
9
InOrder:
4 8 2 5 1 9 6 3 7
PreOrder:
1 2 4 8 5 3 6 9 7
PostOrder:
8 4 5 2 9 6 7 3 1
menampilkan simpul dalam bentuk phon :
      /--7
     /--3
    /  \--6
   /    \--9
  /--1
 /  \--5
/    \--2
/      \--8
/        \--4

```

Gambar 2.1 : Ouput Program Binary Tree

Dari output program Binary Tree Driver dapat dilihat bahwa program akan menampilkan jumlah simpul awal, jumlah simpul jika hanya ada root dan juga jumlah simpul jika simpul 7 ditambahkan. Setelah itu program akan menampilkan hasil dari penelusuran dari ketiga method yaitu preorder, inorder dan juga postorder. Lalu program menampilkan gambar visualisasi dari pohon yang terbentuk.

2.5 Program GraphTraversal_2511531001

2.5.1 Kode Program

```

package Pekan9_2511531001;
import java.util.*;
public class GraphTraversal_2511531001 {

```

```

        private Map <String, List <String>> graph_1001 = new
HashMap <> ();

        //deklarasi dan inisialisasi struktur data graph
menggunakan representasi adjacency list di java

        //Menambahkan edge (graf tak berarah)
        public void addEdge_1001 (String node1_1001, String
node2_1001) { //method untuk menambahkan sisi untuk
menghubungkan satu node dengan node lainnya

            graph_1001.putIfAbsent(node1_1001, new
ArrayList<>()); //memeriksa apakah node1_1001 sudah
terdaftar sebagai key dalam HashMap

            //jika belum maka method ini akan otomatis
menambahkan node1_1001 dengan nilai awal berupa arraylist
yang kosong

            graph_1001.putIfAbsent(node2_1001, new
ArrayList<>()); //memeriksa apakah node2_1001 sudah
terdaftar sebagai key dalam HashMap

            //jika belum maka method ini akan otomatis
menambahkan node2_1001 dengan nilai awal berupa arraylist
yang kosong

            graph_1001.get(node1_1001).add(node2_1001); //mengamb
il daftar tetangga miliki node1_1001 lalu menambahkan
node2_1001 ke dalam daftar tersebut

            //menciptakan jalur node 1 ke node 2

            graph_1001.get(node2_1001).add(node1_1001); //mengamb
il daftar tetangga miliki node2_1001 lalu menambahkan
node1_1001 ke dalam daftar tersebut

            //menciptakan jalur node 2 ke node 1

        }

        //menampilkan graf awal
        public void printGraph_1001() { //method ini
berfungsi untuk menampilkan representasi visual dari
struktur graf

```

```

        System.out.println("Graf Awal (Adjacency
List) : ");
        for (String node_1001 : graph_1001.keySet())
        {//perulangan ini berfungsi untuk menampilkan semua node
        yang ada
            System.out.print(node_1001 + "-> ");
            List<String> neighbors_1001 =
graph_1001.get(node_1001);//mengambil daftar tetangga dari
node yang aktif
            System.out.println(String.join(", ",
neighbors_1001));
        }
        System.out.println();
    }

    //DFS rekursif
    public void dfs_1001 (String start_1001) {//method
    yang menjadi awal dari penelusuran dengan algoritma DFS
        Set <String> visited_1001 = new HashSet <>();
        //membuat himpunan dari node yang sudah dikunjungi dengan
        nama visited_1001
        System.out.println("Penelusuran DFS : ");

        dfsHelper_1001(start_1001,visited_1001);//memanggil
method dfsHelper untuk meneruskan node awal dan objek
visited_1001 agar dapat diisi dan diperiksa
        System.out.println();
    }

    private void dfsHelper_1001 (String current_1001,
Set <String> visited_1001) {// method inilah yang melakukan
penelusuran dfs
        if (visited_1001.contains(current_1001))
return;// percabangan ini bertugas untuk mengecek apakah
node saat ini sudah dikunjungi
        visited_1001.add(current_1001); //jika belum
dikunjungi maka node tersebut ditandai sebagai sudah
dikunjungi dengan memasukkannya ke dalam set

```

```

        System.out.print(current_1001 + " "); // node
        ditampilkan ke layar
        for (String neighbor_1001 :
graph_1001.getDefault(current_1001, new ArrayList<>()))
{ // perulangan ini untuk menelusuri semua tetangga node
  saat ini
      dfsHelper_1001(neighbor_1001,
visited_1001);
    }
  }

  private void bfs_1001 (String start_1001) { // method
  ini berfungsi untuk melakukan penelusuran dengan method bfs

      Set<String> visited_1001 = new HashSet
<>(); // membuat set dengan nama visited_1001 untuk node
yang sudah dikunjungi

      Queue <String> queue_1001 = new
LinkedList<>(); // logika ini untuk memastikan node yang
pertama kali dimasukkan ke antrean akan menjadi node
pertama yang diproses

      queue_1001.add(start_1001);
      visited_1001.add(start_1001); // node awal
yang dimasukkan ke dalam antrian akan langsung ditandai
bahwa sudah dikunjungi

      System.out.println("Penelusuran BFS :");
      while (!queue_1001.isEmpty()) { // perulangan
akan berjalan jika antriannya masih belum kosong

          String current_1001 =
queue_1001.poll(); // mengambil node yang berada paling
depan antrean lalu menghapusnya dari antrean

          System.out.print(current_1001 + " "); //
mencetak dan menampilkan node yang aktif pada saat ini

          for (String neighbor_1001 :
graph_1001.getDefault(current_1001, new ArrayList<>())) {

              if
(!visited_1001.contains(neighbor_1001)) { // mengecek apakah
tetangga tersebut sudah pernah dikunjungi apa belum

```

```

        queue_1001.add(neighbor_1001); // jika belum
        //maka tetangga akan akan
        //dimasukkan ke dalam antrean dan langsung ditandai dengan
        //sudah dikunjungi

        visited_1001.add(neighbor_1001);
    }
}

System.out.println();
}

//main
public static void main (String [] Args) {
    GraphTraversal_2511531001 graph_1001 = new
    GraphTraversal_2511531001();
    //mendeklarasikan graph baru bernama
    graph_1001

    //contoh graf = A-B, A-C, B-D, B-E
    graph_1001.addEdge_1001("A", "B");
    //memberikan sisi untuk menghubungkan node A dengan node B
    graph_1001.addEdge_1001("A", "C");
    //memberikan sisi untuk menghubungkan node A dengan node C
    graph_1001.addEdge_1001("B", "D");
    //memberikan sisi untuk menghubungkan node B dengan node D
    graph_1001.addEdge_1001("B", "E");
    //memberikan sisi untuk menghubungkan node B dengan node E

    //cetak graph awal
    System.out.println("Graf Awal adalah : ");
    graph_1001.printGraph_1001(); // memanggil
    method printGraph untuk menampilkan graph awal

    //lakukan penelusuran
    graph_1001.dfs_1001("A"); //melakukan
    penelusuran dengan method dfs dengan titik awalnya adalah
    node A

```

```
graph_1001.bfs_1001("A");//melakukan
penelusuran dengan method bfs dengan titik awalnya adalah
node A
    }
}
```

Kode Program 2.4 : Program Penelusuran Graph

2.5.2 Output

```
Graf Awal adalah :
Graf Awal (Adjacency List) :
A-> B,C
B-> A,D,E
C-> A
D-> B
E-> B

Penelusuran DFS :
A B D E C
Penelusuran BFS :
A B C D E
```

Gambar 2.2 : Output Program Graph

Output di atas menunjukkan bahwa program akan menampilkan graph awal sebelum dilakukan penelusuran. Setelah itu akan menampilkan hasil dari penelusuran dengan dua method yang berbeda yaitu DFS dan juga BFS. Dapat dilihat juga ada perbedaan dari hasil penelusuran, yaitu setelah dari node B, metode DFS malah mengunjungi node D terlebih dahulu, lalu ke node E dan node C. Berbeda dengan metode BFS, yang terlihat seperti mengunjungi node sesuai dengan urutannya. Hal ini dikarenakan karena DFS akan menelusuri node sampai level terdalamnya sebelum kembali ke root. Sedangkan BFS akan mengunjungi semua node yang ada pada satu level yang sama lalu berpindah ke level setelahnya.

BAB III

KESIMPULAN

Dari praktikum yang telah dilaksanakan, hal yang dapat disimpulkan adalah di antara lain :

1. Pada program pengimplementasian Binary Tree, terdapat 3 metode untuk melaksanakan penelusuran dan tentunya akan menghasilkan jalur penelusuran yang berbeda, yaitu
 - a. Preorder : Metode ini akan mengunjungi root terlebih dahulu lalu mengunjungi left subtree dari binary tree, dan setelahnya mengunjungi right subtree dari binary tree.
 - b. Inorder : Metode ini akan mengunjungi left subtree dari pohon, baru mengunjungi root dan terakhir mengunjungi right subtree.
 - c. Postorder: Metode ini akan mengunjungi left subtree dari pohon terlebih dahulu dan lanjut mengunjungi right subtree, lalu mengunjungi root dari pohon.
2. Pada program pengimplmentasian Graph, terdapat 2 metode penelusuran yang berbeda, yaitu:
 - a. DFS (Depth First Search) : yaitu melakukan penelusuran dari root dan mengunjungi semua node per level hingga mencapai node terdalam dan kemudian mencari node yang belum dikunjungi.
 - b. BFS (Breadth Fist Search) : metode penelusuran yang akan mengunjungi secara “melebar” dimana metode akan mengunjungi root dan berlanjut ke node yang berada pada level selanjutnya dan mengunjungi semua node pada level tersebut sebelum pindah ke level node selanjutnya.

DAFTAR PUSTAKA

- [1] R. Amalia, "Binary Tree adalah: Ciri-ciri, Fungsi, Jenis Struktur Data," *idwebhost.com*, 20 Nov. 2024. <https://idwebhost.com/blog/binary-tree-adalah/> (Diakses pada 12 Jun. 2026).
- [2] "Struktur Data Graph," *Trivusi.web.id*, Jul. 2022. <https://www.trivusi.web.id/2022/07/struktur-data-graph.html> (Diakses pada 12 Jun. 2026).

LAPORAN TUGAS

1. Kode Program

```
package Pekan9_2511531001;

import javax.swing.*;
import javax.swing.border.*;
import java.awt.*;
import java.awt.event.*;
import java.util.*;

public class PetaBandara_2511531001 extends JFrame {

    private final String[] nodes_1001 = {
        "Terminal 1", "Terminal 2", "Terminal 3",
        "Check-In", "Imigrasi", "Gate A",
        "Gate B", "Gate C", "Baggage Claim",
        "Parkir", "Food Court", "Musholla",
        "Toilet", "Informasi", "Pintu Darurat",
        "Zona Penjemputan", "Taxi Service"
    };

    private final Map<String, ArrayList<String>> adjList_1001 =
    new LinkedHashMap<String, ArrayList<String>>();
    private final Map<String, int[]> nodePos_1001 = new
    LinkedHashMap<String, int[]>();
    private final Map<String, String> nodeColor_1001 = new
    HashMap<String, String>();

    private String startNode_1001 = "";
    private String goalNode_1001 = "";

    private JComboBox<String> cbStart_1001;
    private JComboBox<String> cbGoal_1001;
    private GraphPanel_1001 graphPanel_1001;
    private JTextArea resultArea_1001;

    public PetaBandara_2511531001() {
        getContentPane().setForeground(new Color(255, 255, 255));
        initGraph_1001();
        initGUI_1001();
        resetGraph_1001();
    }

    private void initGraph_1001() {
        nodePos_1001.put("Parkir", new int[]{50, 160});
        nodePos_1001.put("Terminal 1", new int[]{150, 320});
        nodePos_1001.put("Check-In", new int[]{250, 160});
        nodePos_1001.put("Imigrasi", new int[]{350, 320});
        nodePos_1001.put("Baggage Claim", new int[]{450, 160});
        nodePos_1001.put("Food Court", new int[]{550, 320});
        nodePos_1001.put("Musholla", new int[]{650, 160});
        nodePos_1001.put("Gate C", new int[]{750, 320});

        nodePos_1001.put("Terminal 2", new int[]{150, 80});
        nodePos_1001.put("Terminal 3", new int[]{450, 80});
    }
}
```

```

        nodePos_1001.put("Gate A", new int[]{650, 80});
        nodePos_1001.put("Pintu Darurat", new int[]{850, 80});

        nodePos_1001.put("Toilet", new int[]{850, 160});
        nodePos_1001.put("Informasi", new int[]{50, 400});
        nodePos_1001.put("Gate B", new int[]{550, 400});
        nodePos_1001.put("Zona Penjemputan", new int[]{850, 320});
        nodePos_1001.put("Taxi Service", new int[]{850, 400});

        for (String n : nodes_1001) adjList_1001.put(n, new
ArrayList<String>());

        addEdge_1001("Parkir", "Terminal 1");
        addEdge_1001("Terminal 1", "Check-In");
        addEdge_1001("Check-In", "Imigrasi");
        addEdge_1001("Imigrasi", "Baggage Claim");
        addEdge_1001("Baggage Claim", "Food Court");
        addEdge_1001("Food Court", "Musholla");
        addEdge_1001("Musholla", "Gate C");
        addEdge_1001("Imigrasi", "Gate C");
        addEdge_1001("Terminal 1", "Terminal 2");
        addEdge_1001("Baggage Claim", "Terminal 3");
        addEdge_1001("Musholla", "Gate A");
        addEdge_1001("Food Court", "Gate B");
        addEdge_1001("Parkir", "Informasi");
        addEdge_1001("Gate C", "Toilet");
        addEdge_1001("Gate C", "Zona Penjemputan");
        addEdge_1001("Zona Penjemputan", "Taxi Service");
        addEdge_1001("Toilet", "Pintu Darurat");
    }

    private void addEdge_1001(String u, String v) {
        adjList_1001.get(u).add(v);
        adjList_1001.get(v).add(u);
    }

    public ArrayList<String> BFS(String start_1001, String
goal_1001) {
        Map<String, String> parent_1001 = new
HashMap<String, String>();
        ArrayList<String> visited_1001 = new
ArrayList<String>();
        LinkedList<String> queue_1001 = new
LinkedList<String>();

        parent_1001.put(start_1001, null);
        queue_1001.add(start_1001);

        while (!queue_1001.isEmpty()) {
            String curr_1001 = queue_1001.poll();
            visited_1001.add(curr_1001);
            nodeColor_1001.put(curr_1001, "visited");

            if (curr_1001.equals(goal_1001)) {
                return buildPath_1001(parent_1001, goal_1001,
visited_1001, "BFS");
            }
        }
    }

```



```

        ArrayList<String>
visited_1001,
        String algo_1001) {
    ArrayList<String> path_1001 = new ArrayList<String>();
    String cur_1001 = goal_1001;
    while (cur_1001 != null) {
        path_1001.add(0, cur_1001);
        cur_1001 = parent_1001.get(cur_1001);
    }
    for (String n : path_1001) nodeColor_1001.put(n, "path");

    StringBuilder sb_1001 = new StringBuilder();
    sb_1001.append("=== Hasil Pencarian
").append(algo_1001).append(" ===\n");
    sb_1001.append("Jalur          :
").append(joinList_1001(path_1001, " -> ")).append("\n");
    sb_1001.append("Node Dikunjungi :
").append(joinList_1001(visited_1001, ", ")).append("\n");
    sb_1001.append("Jumlah Node Dieksplorasi :
").append(visited_1001.size()).append("\n");
    sb_1001.append("Panjang Jalur      :
").append(path_1001.size() - 1).append(" langkah\n");

    resultArea_1001.setText(sb_1001.toString());
    displayGraph_1001();
    return path_1001;
}

private String joinList_1001(ArrayList<String> list_1001,
String sep_1001) {
    StringBuilder sb = new StringBuilder();
    for (int i = 0; i < list_1001.size(); i++) {
        if (i > 0) sb.append(sep_1001);
        sb.append(list_1001.get(i));
    }
    return sb.toString();
}

private void showNoPath_1001(String algo_1001,
ArrayList<String> visited_1001) {
    resultArea_1001.setText("=== Hasil Pencarian " +
algo_1001 + " ===\n"
+ "Jalur tidak ditemukan!\n"
+ "Node Dikunjungi : " + joinList_1001(visited_1001,
", ") + "\n"
+ "Jumlah Node Dieksplorasi : " + visited_1001.size()
+ "\n");
    displayGraph_1001();
}

public void displayGraph_1001() {
    graphPanel_1001.repaint();
}

public void displayPath_1001(ArrayList<String> path_1001) {
    if (path_1001.isEmpty()) {

```

```

        resultArea_1001.append("\n[Tidak ada jalur untuk
ditampilkan]");
    }
}

public void resetGraph_1001() {
    for (String n : nodes_1001) nodeColor_1001.put(n,
"default");
    resultArea_1001.setText("Hasil Pencarian : \nJalur : \nNode
Dikunjungi : \nJumlah Node Dieksplorasi : 0");
    displayGraph_1001();
}

private void initGUI_1001() {
    setTitle("PENCARIAN JALUR BANDARA - BFS & DFS");
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    getContentPane().setLayout(new BorderLayout(8, 8));
    getContentPane().setBackground(new Color(255, 255, 255));

    JLabel header_1001 = new JLabel("PENCARIAN JALUR
MENGGUNAKAN BFS DAN DFS", SwingConstants.CENTER);
    header_1001.setFont(new Font("SansSerif", Font.BOLD,
16));
    header_1001.setForeground(Color.WHITE);
    header_1001.setBackground(new Color(81, 140, 70));
    header_1001.setOpaque(true);
    header_1001.setBorder(BorderFactory.createEmptyBorder(10,
0, 10, 0));

    JPanel controlPanel_1001 = new JPanel(new
FlowLayout(FlowLayout.LEFT, 12, 8));
    controlPanel_1001.setBackground(new Color(230, 158, 87));

    controlPanel_1001.add(makeLabel_1001("Lokasi Awal :"));
    cbStart_1001 = new JComboBox<String>(nodes_1001);
    cbStart_1001.setFont(new Font("SansSerif", Font.PLAIN,
13));
    cbStart_1001.setSelectedItem("Parkir");
    controlPanel_1001.add(cbStart_1001);

    controlPanel_1001.add(makeLabel_1001("Lokasi Tujuan :"));
    cbGoal_1001 = new JComboBox<String>(nodes_1001);
    cbGoal_1001.setFont(new Font("SansSerif", Font.PLAIN,
13));
    cbGoal_1001.setSelectedItem("Gate C");
    controlPanel_1001.add(cbGoal_1001);

    JButton btnBFS_1001 = makeButton_1001("[ BFS ]", new
Color(0, 160, 80));
    btnBFS_1001.addActionListener(new ActionListener() {
        public void actionPerformed(ActionEvent e) {
            resetGraph_1001();
            startNode_1001 = (String)
cbStart_1001.getSelectedItem();
            goalNode_1001 = (String)
cbGoal_1001.getSelectedItem();
            if (startNode_1001.equals(goalNode_1001)) {

```

```

        JOptionPane.showMessageDialog(null, "Lokasi
awal dan tujuan tidak boleh sama!");
        return;
    }
    BFS(startNode_1001, goalNode_1001);
}
});

JButton btnDFS_1001 = makeButton_1001("[ DFS ]", new
Color(60, 100, 200));
btnDFS_1001.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        resetGraph_1001();
        startNode_1001 = (String)
cbStart_1001.getSelectedItem();
        goalNode_1001 = (String)
cbGoal_1001.getSelectedItem();
        if (startNode_1001.equals(goalNode_1001)) {
            JOptionPane.showMessageDialog(null, "Lokasi
awal dan tujuan tidak boleh sama!");
            return;
        }
        DFS(startNode_1001, goalNode_1001);
    }
});

JButton btnReset_1001 = makeButton_1001("[ RESET ]", new
Color(200, 60, 60));
btnReset_1001.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        resetGraph_1001();
    }
});

controlPanel_1001.add(btnBFS_1001);
controlPanel_1001.add(btnDFS_1001);
controlPanel_1001.add(btnReset_1001);

JPanel north_1001 = new JPanel(new BorderLayout());
north_1001.add(header_1001, BorderLayout.NORTH);
north_1001.add(controlPanel_1001, BorderLayout.SOUTH);
getContentPane().add(north_1001, BorderLayout.NORTH);

graphPanel_1001 = new GraphPanel_1001();
graphPanel_1001.setPreferredSize(new Dimension(900,
480));

graphPanel_1001.setBorder(BorderFactory.createTitledBorder(
    BorderFactory.createLineBorder(Color.GRAY), "
VISUALISASI GRAPH ",
    TitledBorder.LEFT, TitledBorder.TOP,
    new Font("Monospaced", Font.BOLD, 12), Color.BLACK));
graphPanel_1001.setBackground(new Color(255,255,255));
getContentPane().add(graphPanel_1001,
BorderLayout.CENTER);

resultArea_1001 = new JTextArea(7, 60);

```

```

        resultArea_1001.setFont(new Font("Monospaced", Font.BOLD,
12));
        resultArea_1001.setBackground(new Color(230, 158, 87));
        resultArea_1001.setForeground(new Color(0, 64, 0));
        resultArea_1001.setEditable(false);

resultArea_1001.setBorder(BorderFactory.createEmptyBorder(6, 8,
6, 8));
        JScrollPane scroll_1001 = new
JScrollPane(resultArea_1001);
        scroll_1001.setToolTipText("");
        scroll_1001.setBorder(new TitledBorder(new LineBorder(new
Color(128, 128, 128)), " Hasil Pencarian ",
TitledBorder.CENTER, TitledBorder.TOP, null, new Color(0, 100,
0)));
        getContentPane().add(scroll_1001, BorderLayout.SOUTH);

        pack();
        setMinimumSize(new Dimension(950, 750));
        setLocationRelativeTo(null);
    }

    private JLabel makeLabel_1001(String text_1001) {
        JLabel lbl = new JLabel(text_1001);
        lbl.setForeground(Color.WHITE);
        lbl.setFont(new Font("SansSerif", Font.BOLD |
Font.ITALIC, 13));
        return lbl;
    }

    private JButton makeButton_1001(String text_1001, Color
bg_1001) {
        JButton btn = new JButton(text_1001);
        btn.setBackground(bg_1001);
        btn.setForeground(new Color(0, 0, 0));
        btn.setFont(new Font("Monospaced", Font.BOLD, 13));
        btn.setFocusPainted(false);
        btn.setBorder(BorderFactory.createEmptyBorder(6, 14, 6,
14));
        return btn;
    }

    class GraphPanel_1001 extends JPanel {

        public void paintComponent(Graphics g) {
            super.paintComponent(g);
            Graphics2D g2_1001 = (Graphics2D) g;

g2_1001.setRenderingHint(RenderingHints.KEY_ANTIALIASING,
RenderingHints.VALUE_ANTIALIAS_ON);

            // Garis putus-putus untuk semua edge
            float[] dashPattern = {8.0f, 5.0f};
            g2_1001.setStroke(new BasicStroke(2f,
BasicStroke.CAP_BUTT, BasicStroke.JOIN_MITER, 10.0f, dashPattern,
0.0f));

```

```

        HashSet<String> drawn_1001 = new HashSet<String>();
        for (String u : adjList_1001.keySet()) {
            for (String v : adjList_1001.get(u)) {
                String key_1001 = u.compareTo(v) < 0 ? u +
                "|" + v : v + "|" + u;
                if (drawn_1001.contains(key_1001)) continue;
                drawn_1001.add(key_1001);

                int[] pu = nodePos_1001.get(u);
                int[] pv = nodePos_1001.get(v);

                String cu = nodeColor_1001.containsKey(u) ?
                nodeColor_1001.get(u) : "default";
                String cv = nodeColor_1001.containsKey(v) ?
                nodeColor_1001.get(v) : "default";

                if (cu.equals("path") && cv.equals("path")) {
                    g2_1001.setColor(new Color(200, 160, 0));
                    // Garis path lebih tebal dan solid
                    g2_1001.setStroke(new BasicStroke(3f));
                } else {
                    g2_1001.setColor(new Color(100, 100,
                100));
                    g2_1001.setStroke(new BasicStroke(2f,
                BasicStroke.CAP_BUTT, BasicStroke.JOIN_MITER, 10.0f, dashPattern,
                0.0f));
                }

                g2_1001.drawLine(pu[0], pu[1], pv[0], pv[1]);
            }
        }

        // Node hanya berupa teks, tanpa bentuk visual
        for (String n : nodes_1001) {
            int[] p = nodePos_1001.get(n);
            String col_1001 = nodeColor_1001.containsKey(n) ?
            nodeColor_1001.get(n) : "default";

            Color textColor;
            if (col_1001.equals("visited")) {
                textColor = new Color(200, 100, 0);
            } else if (col_1001.equals("path")) {
                textColor = new Color(0, 150, 50);
            } else {
                textColor = new Color(30, 30, 30);
            }

            g2_1001.setColor(textColor);
            g2_1001.setFont(new Font("SansSerif", Font.BOLD,
            11));

            FontMetrics fm_1001 = g2_1001.getFontMetrics();
            String[] parts_1001 = n.split(" ", 2);
            if (parts_1001.length == 2) {
                int w1 = fm_1001.stringWidth(parts_1001[0]);
                int w2 = fm_1001.stringWidth(parts_1001[1]);
                g2_1001.drawString(parts_1001[0], p[0] - w1 /
            2, p[1] - 4);

```

```

        g2_1001.drawString(parts_1001[1], p[0] - w2 /
2, p[1] + 10);
    } else {
        int w = fm_1001.stringWidth(n);
        g2_1001.drawString(n, p[0] - w / 2, p[1] +
4);
    }
}

drawLegend_1001(g2_1001);
}

private void drawLegend_1001(Graphics2D g2) {
    int lx = 10, ly = getHeight() - 70;
    g2.setFont(new Font("SansSerif", Font.BOLD, 11));
    g2.setColor(Color.BLACK);
    g2.drawString("Keterangan:", lx, ly);

    g2.setFont(new Font("SansSerif", Font.PLAIN, 10));
    g2.setColor(new Color(30, 30, 30));
    g2.drawString("• Belum Dikunjungi", lx, ly + 18);

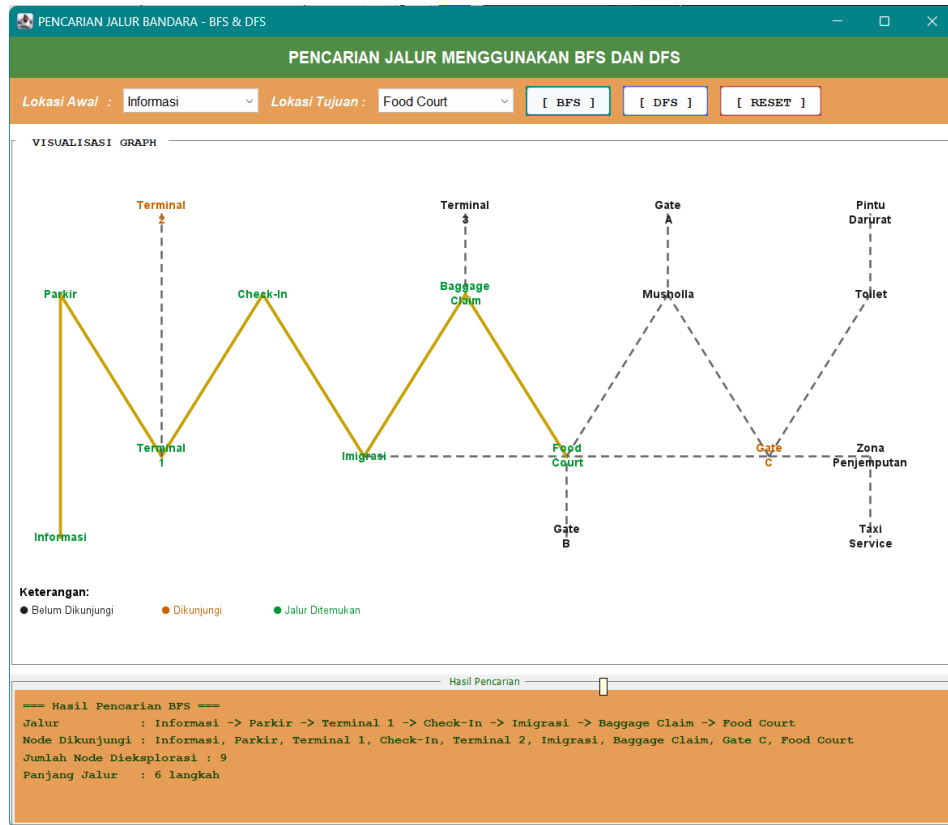
    g2.setColor(new Color(200, 100, 0));
    g2.drawString("• Dikunjungi", lx + 140, ly + 18);

    g2.setColor(new Color(0, 150, 50));
    g2.drawString("• Jalur Ditemukan", lx + 250, ly +
18);
}
}

public static void main(String[] args) {
    try {
        UIManager.setLookAndFeel(UIManager.getSystemLookAndFeelClassName(
));
    } catch (Exception ignored) {}
    SwingUtilities.invokeLater(new Runnable() {
        public void run() {
            new PetaBandara_2511531001().setVisible(true);
        }
    });
}
}

```

2. Output



3. Penjelasan

a. Deskripsi Peta

Peta ini merepresentasikan tata letak sebuah bandara yang terdiri dari berbagai fasilitas penting seperti terminal, gerbang keberangkatan, area check-in, imigrasi, hingga zona penjemputan. Setiap lokasi digambarkan sebagai titik (node) yang saling terhubung melalui jalur corridors (edge). Peta ini dirancang khusus dengan bentuk zigzag dan sebuah jalur pintas (shortcut) untuk memperlihatkan perbedaan cara kerja algoritma BFS dan DFS secara visual.

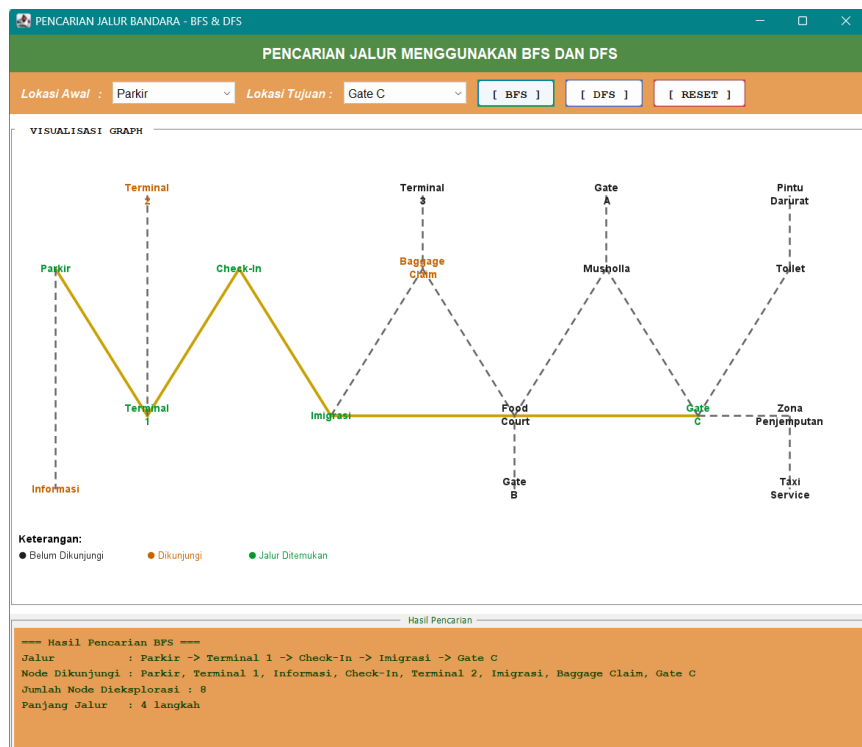
b. Jumlah Node dan Edge

- Jumlah Node : 17 lokasi (Terminal 1, 2, 3, Check-In, Imigrasi, Gate A, B, C, Baggage Claim, Parkir, Food Court, Musholla, Toilet, Informasi, Pintu Darurat, Zona Penjemputan, Taxi Service).
- Jumlah Edge : 17 jalur penghubung antar lokasi.

c. Hasil BFS

Contoh penelusuran dengan start : Parkir dan goal: Gate C

- Jalur : Parkir → Terminal 1 → Check-In → Imigrasi → Gate C
- Panjang Jalur : 4 langkah
- Node Dikunjungi : 8 node (Parkir, Terminal 1, Informasi, Check-In, Terminal 2, Imigrasi, Baggage Claim, Gate C)
- BFS langsung menemukan jalur terpendek karena menyebar secara merata lapis demi lapis.
- Output :

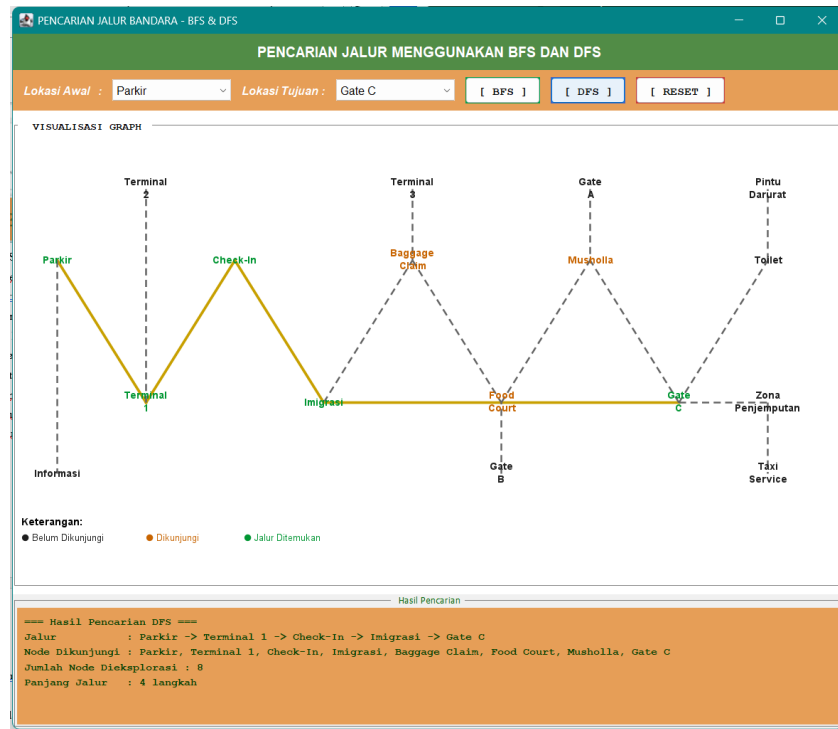


d. Hasil DFS

Contoh penelusuran dengan start : Parkir dan goal: Gate C

- Jalur : Parkir → Terminal 1 → Check-In → Imigrasi → Gate C
- Panjang Jalur : 4 langkah
- Node Dikunjungi : 11 node (Parkir, Terminal 1, Check-In, Imigrasi, Baggage Claim, Food Court, Musholla, Gate A, Gate B, Terminal 3, Gate C)
- DFS menemukan jalur yang sama, tetapi ia "tersesat" lebih dulu menyusuri cabang dalam (Musholla, Gate A, Gate B, Terminal 3) sebelum akhirnya kembali dan menemukan Gate C.

- Output :



e. Kesimpulan Perbandingan BFS dan DFS

- BFS menjamin jalur terpendek karena menjelajah node per level secara melebar, sehingga lebih efisien dalam jumlah node yang dikunjungi (8 node).
- DFS menjelajah secara mendalam ke satu cabang terlebih dahulu sebelum berpindah, sehingga sering mengunjungi lebih banyak node (11 node) meski akhirnya menemukan jalur yang sama.
- BFS cocok untuk mencari rute terdekat, sedangkan DFS cocok untuk menjelajah seluruh area hingga ke ujung
- Pada peta ini, perbedaan keduanya terlihat jelas dari jumlah node yang diwarnai oranye (dikunjungi) saat tombol ditekan.